RESULTS

Exam Results

884 / 1000

PASSED

53 of 60 correct (88%) · Pass mark: 720

Domain Breakdown

D1	Agentic Architecture & Orchestration	13/14 (93%)	+251pts
D2	Tool Design & MCP Integration	10/11 (91%)	+164pts
D3	Claude Code Configuration & Workflows	9/12 (75%)	+150pts
D4	Prompt Engineering & Structured Output	11/12 (92%)	+183pts
D5	Context Management & Reliability	10/11 (91%)	+136pts

HOW SCORING WORKS

Each domain's percentage correct is multiplied by its weight and scaled to 1000 points. For example, scoring 80% in Agentic Architecture (27% weight) contributes 0.80 × 0.27 × 1000 = 216 points. The pass mark is 720/1000 (72%). If a domain has no questions in the selected scenarios, its weight is redistributed proportionally among assessed domains.

The real exam reports a scaled score from 100 to 1,000 plus your percent-correct by domain, so treat the percentages here as a readiness gauge rather than a prediction of your scaled score.

HIDE DETAIL | TAKE ANOTHER EXAM

Question-by-Question Review

ALL 60 | CORRECT 53 | INCORRECT 7 | SKIPPED 0

Q01 · D4 4.3 STRUCTURED-OUTPUT / ENUM-FIELDS q-4-3-004 · Content Moderation and Classification System CORRECT

The moderation system's classification schema has a 'category' field defined as a free-text string. Auditors find 47 different category values in production data, including 'hate speech', 'Hate Speech', 'hate-speech', 'hateful content', and 'hate_speech' — all intended to be the same category. This makes downstream analytics and routing unreliable. What is the best schema fix?

A Add a post-processing normalisation step that maps all variations to canonical category names

Post-processing normalisation adds complexity and requires constant maintenance as new variations appear. The schema should prevent the problem at the source rather than patching it downstream.

B Change 'category' from free-text to an enum with values like 'hate_speech', 'spam', and 'harassment', plus an 'other' option.

Enum fields constrain the model to predefined values, eliminating spelling and formatting variations. Including 'other' as an enum value handles edge cases without allowing free-text drift. With strict: true, the schema guarantees only valid enum values are returned.

D Add few-shot examples showing the correct category formatting for each type

Few-shot examples improve consistency but cannot guarantee exact string matching. Schema enums are the correct mechanism for constraining categorical output to a fixed set of values.

C Add detailed instructions to the prompt listing the exact category names and their capitalisation so the model always emits the canonical string.

Prompt instructions are probabilistic. The model may still produce variations despite instructions. Schema-level enforcement via enums is deterministic and cannot be overridden.

WHERE THIS COMES FROM

Lesson 4.2: Structured Output with Tool Use (Enum schema)

Q02 · D2 2.2 STRUCTURED-ERROR-RESPONSES / TRANSIENT-ERROR q-2-2-004 · Customer Support Resolution Agent CORRECT

A customer support MCP server receives a request to look up a customer record, but the database connection times out after 30 seconds. The server needs to communicate this failure to the agent. What is the correct way to structure the MCP error response?

D Throw an unhandled exception to let the MCP framework propagate the error to the agent automatically.

Unhandled exceptions produce generic error messages without structured metadata. The agent cannot distinguish error types or determine retry behaviour from an unhandled exception.

B Set isError to true and return a structured response with errorCategory: 'transient', isRetryable: true, and a message describing the timeout.

A database timeout is a transient error — the database may be available on the next attempt. Setting isError: true signals failure, errorCategory: 'transient' classifies it correctly, and isRetryable: true tells the agent it is safe to retry.

C Set isError to true and return errorCategory: 'business', isRetryable: false, since the operation could not be completed.

A database timeout is not a business error. Business errors represent policy violations (such as exceeding a refund limit) that will never succeed on retry. A timeout is transient — the database may recover, making retry appropriate.

A Return an empty result with no error flag, and let the agent infer the failure from the missing data.

Returning empty results without error metadata makes the failure indistinguishable from a valid empty result (e.g. customer not found). The agent cannot determine whether to retry or accept the result.

WHERE THIS COMES FROM

Lesson 2.2: Structured Error Responses (Transient errors)

MCP: Tools Specification ↗

Q03 · D1 1.5 AGENT-SDK-HOOKS / POSTTOOLUSE-HOOKS q-1-5-012 · Customer Support Resolution Agent CORRECT

A customer support agent must redact credit card numbers from its responses before they reach the customer. The current system prompt instructs the agent to replace card numbers with asterisks, but QA testing reveals that 6% of responses still contain unredacted card numbers. What guardrail approach should the architect use?

D Remove all tools that might return credit card numbers from the agent's allowedTools

Removing tools that return card numbers would eliminate the agent's ability to look up order and payment information, which is core to its customer support function. The correct approach is to keep the tools but redact sensitive data from their results.

A Rewrite the system prompt with more explicit redaction instructions and add few-shot examples of correct redaction

The current prompt already instructs redaction but fails 6% of the time. Strengthening prompts may reduce the failure rate but cannot eliminate it. Leaking credit card numbers is a data security issue that demands deterministic enforcement.

B Add a PostToolUse hook that regex-redacts credit card numbers from tool results before the model sees them.

A PostToolUse hook runs deterministic code (regex pattern matching) on tool results before the model sees them. This ensures credit card numbers are redacted with 100% reliability, regardless of model behaviour. This is the correct guardrail for data security requirements.

C Implement a PreToolUse hook that blocks any tool call containing a credit card number in its parameters

PreToolUse hooks intercept outgoing tool calls, not incoming results. The credit card numbers appear in tool results (data returned from backend systems), not in tool call parameters. PostToolUse is the correct hook type for redacting data from results.

WHERE THIS COMES FROM

Lesson 1.5: Agent SDK Hooks (PostToolUse hooks)

Anthropic: Agent SDK Hooks ↗

Q04 · D4 4.5 BATCH-PROCESSING / CUSTOM-ID q-4-5-004 · CI/CD Pipeline Integration CORRECT

Your CI/CD pipeline processes code reviews using the Message Batches API. After a batch completes, you need to match each review result back to the corresponding pull request. What is the correct mechanism for correlating batch results with their original requests?

D Submit each pull request as a separate batch of one item to maintain correlation through the batch ID

Submitting individual batches defeats the purpose of batch processing and does not leverage the custom_id mechanism designed for exactly this use case.

B Rely on the batch API returning results in the same order they were submitted

Batch results are not guaranteed to return in submission order. The custom_id field is the correct correlation mechanism.

C Use the custom_id field in each batch request to store the pull request identifier, then match results by custom_id

The custom_id field exists specifically for correlating batch request/response pairs. Setting it to the pull request identifier provides reliable, direct correlation without relying on result ordering or content parsing.

A Parse the review content to identify which pull request it refers to based on file names mentioned in the review

Content parsing is fragile and error-prone. The Batches API provides a built-in mechanism for result correlation.

WHERE THIS COMES FROM

Lesson 4.5: Batch Processing and Prompt Optimisation (custom_id matching)

Anthropic: Message Batches API ↗

Q05 · D3 3.6 CICD-INTEGRATION / SESSION-ISOLATION q-3-6-002 · CI/CD Pipeline Integration CORRECT

A CI/CD pipeline uses Claude Code to generate tests in one step and then review the same code in a subsequent step. The team notices that the review step rarely finds issues with the generated tests, but human reviewers consistently find problems. Both CI steps use the -p flag correctly. What is the most likely cause and fix?

D The generation and review steps must run as independent sessions with no shared context, so the review instance approaches the code without the original reasoning bias

The same Claude session (or a session with carried-over context) that generated code is less effective at reviewing its own changes because it retains the reasoning context from generation. An independent review instance without that context is more likely to catch issues, matching what human reviewers find. Session context isolation for independent reviews is a key CI integration concept.

C The review step runs the same model tier as generation, which is not capable enough to critique its own output; upgrade the review step to a more powerful model

Model capability is not the bottleneck. A more powerful model still inherits self-review bias when it runs in the generating session's context, because it retains the reasoning it used to justify the code. The pattern here (a review that passes work human reviewers reject) points to shared reasoning context, which is fixed by running the review as an independent session with no shared context, not by upgrading the model.

A The review step needs --output-format json to produce structured findings that can be parsed programmatically

The output format affects how findings are reported, not whether issues are detected. The problem is that the review misses issues entirely, not that findings are in the wrong format.

B The CLAUDE.md file lacks testing standards and review criteria, so the review has no project-specific context

While documenting testing standards in CLAUDE.md is important, it would affect the quality of both generation and review. The specific pattern here — review missing issues that humans catch — points to self-review bias, not missing context.

WHERE THIS COMES FROM

Lesson 3.6: CI/CD Integration (Session context isolation)

Q06 · D4 4.2 FEW-SHOT-PROMPTING / FEW-SHOT-COVERAGE q-4-2-001 · Structured Data Extraction Pipeline CORRECT

Your extraction pipeline correctly identifies data in structured tables but returns empty fields when the same information appears in narrative paragraphs. Detailed instructions specify all required fields. What should you try first?

A Increase the model's context window to process more of the document

The model already finds data in tables. The issue is not context size but inconsistent handling of different document structures.

D Add a post-processing retry that re-extracts any empty fields

Retrying without better guidance will produce the same empty results. The model needs examples of correct extraction from narrative text.

B Add few-shot examples showing correct extraction from both structured tables and narrative paragraphs

Few-shot examples demonstrating correct handling of varied document structures directly address inconsistent extraction quality across formats.

C Add a pre-processing step to convert all narrative text into tables

This adds unnecessary complexity. The model should handle varied formats — few-shot examples teach it how.

WHERE THIS COMES FROM

Lesson 4.2: Few-Shot Prompting (Few-shot covering varied input)

Lesson 4.2: Structured Output with Tool Use (What tool_use does not enforce)

Q07 · D2 2.5 BUILT-IN-TOOLS / GREP-VS-GLOB q-2-5-008 CORRECT

A developer is investigating a data inconsistency in the platform codebase. They need to find all files that reference a specific Snowflake table name 'fact_revenue_daily' and then check the directory structure for related migration files. Which tool sequence is correct?

B Read all SQL files in the project to search for the table name, then Read all migration files.

Reading all files upfront is a context-budget anti-pattern. It wastes tokens on irrelevant files and is exactly the approach the exam penalises.

C Grep for 'fact_revenue_daily' to find all content references, then Glob for '**/migrations/*.sql' to find migration files by path pattern.

Grep searches file contents — correct for finding code that references the table name. Glob matches file paths — correct for finding migration files by directory pattern. This is the optimal two-step sequence using each tool for its strength.

D Grep for 'fact_revenue_daily' for both steps — first to find references, then to find migration files containing the table.

Using Grep for both steps would find migration files that mention the table, but would miss migration files related to the table that do not contain its name (e.g. schema migrations, index changes). Glob is the correct tool for path-based file discovery.

A Glob for '**/*fact_revenue_daily*' to find references, then Glob for '**/migrations/**' to find migration files.

Glob matches file paths by name, not file contents. It would find files named after the table (unlikely) but miss all files that reference the table name in their code.

WHERE THIS COMES FROM

Lesson 2.5: Built-in Tools (Grep vs Glob)

Q08 · D2 2.1 TOOL-SCHEMA-DESIGN / SYSTEM-PROMPT q-2-1-007 · Customer Support Resolution Agent CORRECT

A support agent has `process_refund` and `adjust_billing` tools with clearly differentiated descriptions, yet still occasionally calls `adjust_billing` for refund requests. The system prompt contains the instruction: 'For any billing-related concerns, prioritise resolving them quickly using available billing tools.' What is the root cause?

A The tool descriptions are still too similar and need further differentiation with explicit boundary statements.

The descriptions were already improved and clearly differentiate the tools. The misrouting persists because a system prompt instruction is creating an unintended association between the keyword 'billing' in refund requests and the billing tool.

B The system prompt's 'billing' wording steers refund requests to adjust_billing, overriding the improved tool descriptions.

System prompt wording can silently override well-written tool descriptions. The phrase 'billing-related concerns' matches refund requests that mention billing, causing the model to favour adjust_billing even when process_refund is the correct tool. Reviewing system prompts for conflicts after updating tool descriptions is essential.

C The model needs few-shot examples demonstrating that refund requests should use process_refund, not adjust_billing.

Few-shot examples add token overhead and treat the symptom rather than the cause. The root issue is the system prompt instruction creating a conflicting signal that needs to be resolved directly.

D The two tools should be consolidated into a single financial_operation tool that handles both refunds and billing adjustments.

Consolidation eliminates the useful distinction between refunds and billing corrections. The fix is to correct the system prompt instruction, not to merge tools that serve different purposes.

WHERE THIS COMES FROM

Lesson 2.1: Tool Interface Design (System prompt interactions)

Q09 · D5 5.2 ESCALATION-AMBIGUITY / EXPLICIT-ESCALATION q-5-2-001 · Customer Support Resolution Agent CORRECT

A customer support agent achieves only 55% first-contact resolution, well below the 80% target. Logs show it escalates straightforward damage replacement cases while attempting to autonomously handle complex policy exception requests. What is the most effective improvement?

C Add explicit escalation criteria to the system prompt with few-shot examples demonstrating when to escalate versus resolve autonomously

This directly addresses unclear decision boundaries with concrete examples. It is the proportionate first response before adding infrastructure.

D Deploy a separate classifier model trained on historical tickets to predict which requests need escalation

This is over-engineered, requiring labelled data and ML infrastructure when prompt optimisation has not been tried first.

B Have the agent self-report a confidence score (1-10) and automatically route to humans when confidence falls below a threshold

LLM self-reported confidence is poorly calibrated — the agent is already incorrectly confident on hard cases and uncertain on easy ones.

A Implement sentiment analysis to detect customer frustration and automatically escalate when negative sentiment exceeds a threshold

Sentiment does not correlate with case complexity. This would escalate frustrated but straightforward cases and miss calm but complex ones.

WHERE THIS COMES FROM

Lesson 5.2: Escalation and Ambiguity Resolution (Explicit escalation criteria)

Lesson 5.2: Escalation and Ambiguity Resolution (Valid escalation triggers)

Q10 · D4 4.1 SYSTEM-PROMPTS / SEVERITY-EXAMPLES q-4-1-002 · CI/CD Pipeline Integration CORRECT

Your Claude Code review prompt for the pipeline's polyglot codebase classifies severity using prose descriptions like 'critical means the code is dangerous' and 'minor means the code is slightly suboptimal'. Developers complain that identical code patterns receive different severity ratings across runs. What is the most effective improvement?

A Replace prose severity descriptions with concrete TypeScript code examples for each severity level.

Severity calibration with concrete code examples produces consistent classification across invocations. Prose descriptions are inherently ambiguous and leave the model to interpret severity differently each time.

B Add a confidence threshold so only findings above 90% confidence are reported, filtering out uncertain severity ratings

Self-reported confidence scores are poorly calibrated. Confidence-based filtering does not fix the root cause, which is ambiguous severity criteria. Explicit code examples are the correct fix.

D Add a second model pass that re-evaluates each finding's severity to catch inconsistencies

A second pass using the same vague prose criteria will produce the same inconsistency. The criteria themselves must be improved with concrete code examples before adding verification layers.

C Lower the model temperature to 0 to ensure deterministic severity ratings

While lower temperature reduces randomness, it does not address the fundamental problem: the model has no concrete reference for what each severity level looks like. Code examples provide that reference.

WHERE THIS COMES FROM

Lesson 4.1: System Prompts with Explicit Criteria (Severity calibration)

Q11 · D1 1.5 AGENT-SDK-HOOKS / PRETOOLUSE-ENFORCEMENT q-1-5-015 · Customer Support Resolution Agent CORRECT

A customer support agent handles account deletion requests. The company policy requires explicit human approval before any account is permanently deleted. Currently, the agent sometimes processes deletions autonomously when the customer is insistent. What approval mechanism should the architect implement?

A Add a system prompt instruction: 'Always get manager approval before processing account deletions'

Prompt instructions are probabilistic. The question states the agent 'sometimes processes deletions autonomously when the customer is insistent,' showing that prompts alone are insufficient for irreversible operations that require 100% human approval.

D Implement a PostToolUse hook that reviews the deletion after it has been processed and reverts it if no approval was recorded

PostToolUse hooks run after execution. Account deletion may be irreversible or difficult to undo. Prevention via a PreToolUse hook is required, not post-execution rollback.

C Remove the delete_account tool from the agent entirely and require customers to call a separate phone line

This creates a poor customer experience and eliminates the agent's ability to facilitate the process. The correct approach is to keep the tool but gate it behind human approval, providing a seamless escalation flow.

B Implement a PreToolUse hook on the delete_account tool that pauses execution and routes the request to a human approval queue before proceeding

A PreToolUse hook on the delete_account tool intercepts the call before execution and enforces the human approval requirement deterministically. The deletion cannot proceed until a human explicitly approves it, regardless of how insistent the customer is.

WHERE THIS COMES FROM

Lesson 1.5: Agent SDK Hooks (PreToolUse hooks)

Lesson 1.4: Workflow Enforcement and Handoff (Human approval gates)

Q12 · D5 5.5 HUMAN-REVIEW-CALIBRATION / STRATIFIED-SAMPLING q-5-5-002 · Structured Data Extraction Pipeline CORRECT

A legal document extraction pipeline has been running for three months with calibrated confidence thresholds. Extractions above 90% confidence are auto-approved without human review, saving 60% of reviewer time. A new client submits contracts written in an unusual two-column legal format the system has not encountered before. The system reports 92% confidence on these extractions. What is the appropriate safeguard?

C Raise the confidence threshold from 90% to 98% for all document types to account for novel formats

Raising the threshold globally penalises well-understood document types that are genuinely accurate at 90%+. This wastes reviewer capacity on documents that do not need review, while still not guaranteeing detection of format-specific issues.

A Maintain stratified random sampling of high-confidence extractions across document types to catch the new format's error rate.

Stratified random sampling is designed precisely for this scenario: detecting novel error patterns in high-confidence extractions that slip past threshold-based routing. By sampling across document types, the new format's errors would surface during routine verification even though the confidence scores appear acceptable.

D Add a document format classifier that flags unknown formats for mandatory human review before any auto-approval

While not wrong in principle, this adds ML infrastructure complexity that stratified sampling already handles. A format classifier requires training data and maintenance, whereas ongoing stratified sampling detects novel error patterns without additional models.

B Trust the calibrated confidence threshold since it was validated against labelled data and the system reports 92% confidence, which exceeds the 90% auto-approval threshold

Confidence calibration is only valid for document types represented in the calibration data. A novel format the system has never encountered can produce miscalibrated confidence scores — the model may be systematically overconfident because it does not recognise its own unfamiliarity with the format.

WHERE THIS COMES FROM

Lesson 5.5: Human Review and Confidence Calibration (Stratified sampling)

Q13 · D4 4.4 VALIDATION-RETRY / RETRY-BOUNDARY q-4-4-001 · Structured Data Extraction Pipeline CORRECT

Your extraction pipeline validates that line item amounts sum to the stated total. For Document A, the sum is £450 but the stated total is £500. For Document B, the 'department' field is missing entirely from the source text. Which retry strategy is correct?

D Skip retries for both and flag them for human review

Document A's discrepancy is likely fixable with error feedback. Skipping the retry wastes the model's self-correction capability.

B Retry Document A with the discrepancy error; flag Document B for human review since the information is absent from the source

Document A has a fixable discrepancy (the model may have missed a line item). Document B has genuinely absent information — retries are ineffective.

C Retry both documents with the same prompt since extraction is non-deterministic

Non-determinism does not create information that is absent. Document A may benefit from retry; Document B will not.

A Retry both: send the validation error for Document A and instruct the model to find the missing department for Document B

Document B's department is absent from the source. Retrying will not make information appear that does not exist.

WHERE THIS COMES FROM

Lesson 4.4: Validation, Retry, and Feedback Loops (Retry effectiveness boundary)

Q14 · D1 1.2 ORCHESTRATION-PATTERNS / ISOLATION q-1-2-010 · Content Moderation and Classification System CORRECT

The moderation coordinator dispatches posts to specialist subagents. The text classifier has been extended to also handle image analysis: its prompt is now 2,500 tokens, its tool list has grown to 12 tools, and accuracy has dropped on both text and image tasks. What principle was violated and what is the fix?

A The text classifier's context window is too small — upgrade to a model with a larger context window to handle the expanded prompt

Context window size is not the issue. Combining unrelated responsibilities in a single agent degrades performance because the model must juggle competing instructions and tools. A larger context window does not fix scope creep.

B Isolation was violated; restore the text classifier to text-only work with scoped tools and keep image analysis separate.

Each subagent should have a focused responsibility and a scoped tool set (4-5 tools maximum). Combining text and image analysis in one agent violates isolation, bloats the prompt, and confuses the model. Restoring separate specialists with scoped tools improves accuracy for both tasks.

C Add better instructions to the combined subagent's prompt to clarify when it should use text tools versus image tools

Better instructions do not address the fundamental problem of an overloaded agent. Twelve tools and 2,500 tokens of instructions degrade performance regardless of instruction quality. The fix is architectural, not instructional.

D Split the combined agent into three narrower specialists: text sentiment, text policy, and image analysis

Over-splitting into too many narrow specialists adds coordination overhead without clear benefit. The original architecture had a text classifier and an image analyser, which is the correct level of decomposition. Restoring that structure is the fix.

WHERE THIS COMES FROM

Lesson 1.2: Multi-Agent Orchestration (Isolation principle)

Lesson 1.4: Workflow Enforcement and Handoff (Scoped tool access)

Q15 · D2 2.2 STRUCTURED-ERROR-RESPONSES / ACCESS-VS-EMPTY q-2-2-001 · Customer Support Resolution Agent CORRECT

A tool returns an empty array after a customer lookup. The agent retries 3 times, then escalates to a human agent. Analysis shows the customer's account simply does not exist. What is the root cause of this wasted effort?

C The escalation threshold is too aggressive — the agent should exhaust more retries before involving a human.

The problem is not the escalation threshold. The problem is that the agent retries at all. A valid empty result requires no retry and no escalation.

D The system prompt should instruct the agent not to retry customer lookups.

Hard-coding retry rules per tool in the system prompt is brittle. The proper fix is structured error metadata that tells the agent whether the result is retryable.

A The retry limit is too low — increasing to 5 retries would resolve the issue.

More retries make the problem worse. The tool succeeded — it found no matching customer. Retrying a successful query with no matches will never produce different results.

B The tool does not distinguish between access failures and valid empty results, so the agent treats 'no matches' as a retriable failure.

The tool successfully queried the data source and found no matches. This is a valid empty result, not an access failure. The agent's recovery logic cannot distinguish the two, causing it to retry when it should accept the result.

WHERE THIS COMES FROM

Lesson 2.2: Structured Error Responses (Access failure vs empty result)

Q16 · D3 3.3 PATH-SPECIFIC-RULES / GLOB-PATTERNS q-3-3-001 · CI/CD Pipeline Integration CORRECT

A codebase has test files co-located with source files throughout 50+ directories (e.g., Button.test.tsx next to Button.tsx). The team wants all tests to follow the same conventions regardless of location. What is the most maintainable approach?

A Create a .claude/rules/ file with frontmatter paths: ["**/*.test.tsx", "**/*.test.ts"] holding the test conventions

Glob patterns in .claude/rules/ match files by pattern across the entire codebase. The conventions load automatically when editing any test file, regardless of directory. No maintenance burden as new test files are added.

C Add all test conventions to the root CLAUDE.md file

Root CLAUDE.md loads for every session. Test conventions would consume tokens even when editing non-test files like API handlers or database models.

B Place a CLAUDE.md file in every directory containing test files

With 50+ directories, this creates massive duplication and maintenance burden. Every new directory with tests would need a copy.

D Create a skill in .claude/skills/ with a paths frontmatter matching the test files so it auto-activates whenever a test file is edited

Skills can auto-activate via a paths frontmatter, but they load on demand as a task-style workflow, not as always-in-context guidance. Test conventions must shape every edit to a matching file, which is what .claude/rules/ provides: rules load into context automatically when Claude reads a matching file, with no invocation step.

WHERE THIS COMES FROM

Lesson 3.3: Path-Specific Rules (Path-specific rules)

Lesson 3.3: Path-Specific Rules (Practical examples)

Q17 · D3 3.4 PLAN-MODE-EXECUTION / PLAN-MODE q-3-4-006 CORRECT

A developer extracting the notification subsystem from a monolith faces 12 cross-module dependencies, 3 messaging patterns, and several valid extraction strategies. They start in direct execution mode. After moving 8 files, the chosen approach breaks circular dependencies with the user-profile module. What should they have done differently?

C Used direct execution but processed only 2 files at a time to catch problems earlier

Smaller batches in direct execution do not address the fundamental issue: the developer chose a strategy without understanding the full dependency landscape. The circular dependency would still be discovered mid-migration, just slightly sooner.

A Used direct execution but with more detailed upfront instructions specifying how to handle each dependency

The developer did not know about the circular dependency with the user-profile module upfront. More detailed instructions cannot account for undiscovered dependencies. The codebase needed exploration first.

D Delegated the entire extraction to a subagent to isolate the risk

Delegating to a subagent does not change the outcome if the subagent also uses direct execution without exploring dependencies. The mode of execution (plan vs direct) matters more than who executes it.

B Used plan mode to map the 12 dependencies and evaluate extraction strategies before committing

A subsystem with 12 dependencies, multiple messaging patterns, and several valid strategies is a textbook plan mode scenario. Plan mode would have mapped the dependency graph and revealed the circular dependency with user-profile before any files were moved, avoiding costly rework.

WHERE THIS COMES FROM

Lesson 3.4: Plan Mode vs Direct Execution (Plan mode for migrations)

Lesson 3.4: Plan Mode vs Direct Execution (Recognising complexity)

Q18 · D3 3.6 CICD-INTEGRATION / SESSION-ISOLATION q-3-6-005 · CI/CD Pipeline Integration CORRECT

A CI/CD pipeline runs three Claude Code steps in sequence: (1) generate a changelog from git commits, (2) review the changelog for accuracy, and (3) check for breaking changes. The team notices that step 2 never flags inaccuracies and step 3 misses obvious breaking changes that the changelog omits. What is the root cause?

B The CLAUDE.md file does not contain changelog formatting standards, so the review step has no criteria to evaluate against

Missing formatting standards would affect the quality of the generated changelog, not the review's ability to catch inaccuracies. The pattern of reviews consistently missing issues points to shared-context bias, not missing criteria.

A The three steps share session context, so steps 2 and 3 inherit step 1's reasoning instead of judging the changelog

When review and analysis steps share context with the generation step, they inherit the original reasoning and are less likely to identify gaps or errors. Independent sessions force each step to evaluate the changelog from scratch without

bias from the generation reasoning. Session context isolation is critical for CI/CD review pipelines.

D The steps need to use --output-format json so that each step can parse the previous step's structured output

Output format affects how results are structured, not whether reviews are thorough. JSON output would help with parsing but would not address the fundamental issue of review steps retaining generation context bias.

C The -p flag is not being used, causing each step to wait for interactive input

If -p were missing, the pipeline would hang rather than produce output that misses issues. The steps are producing output (the changelog is generated, reviews complete), but the reviews are ineffective.

WHERE THIS COMES FROM

Lesson 3.6: CI/CD Integration (Session context isolation)

Q19 · D5 5.5 HUMAN-REVIEW-CALIBRATION / AGGREGATE-METRICS-TRAP q-5-5-004 · Structured Data Extraction Pipeline CORRECT

A structured data extraction system processes invoices, purchase orders, and contracts. The monitoring dashboard shows 97% overall extraction accuracy and the team considers the system production-ready. A detailed audit reveals: invoices 99.5% accuracy (80% of volume), purchase orders 98% accuracy (15% of volume), contracts 72% accuracy (5% of volume). What does this reveal and what action is required?

B The aggregate metric is masking a severe per-type disparity. Give contracts mandatory review and report per-type metrics.

This is a textbook case of aggregate metrics hiding per-type problems. 97% overall is misleading because high-volume invoice accuracy (99.5%) overwhelms the poor contract performance (72%). Per-type metrics would have surfaced this immediately. Contracts need different treatment until the extraction quality is acceptable.

A The system is performing well. 72% on contracts is acceptable since contracts represent only 5% of volume and barely affect the overall metric

Acceptability depends on the business impact, not volume percentage. Contract extraction errors may have severe financial or legal consequences despite low volume. A 28% error rate on any document type is significant.

C The training data needs rebalancing so that contracts represent a larger proportion, which will naturally improve contract accuracy

LLMs used for extraction are not retrained on production data this way. The issue is monitoring visibility and operational response, not model training data distribution.

D The 97% threshold should be raised to 99% overall to force improvement across all document types

Raising the aggregate threshold does not address per-type disparities. Even at 99% overall, contract accuracy could remain low if invoice volume continues to dominate the metric. The solution is per-type metrics, not a higher aggregate bar.

WHERE THIS COMES FROM

Lesson 5.5: Human Review and Confidence Calibration (Aggregate metrics trap)

Q20 · D1 1.2 ORCHESTRATION-PATTERNS / ISOLATION q-1-2-009 · Content Moderation and Classification System CORRECT

The content moderation system uses a hub-and-spoke architecture with a coordinator that routes posts to specialist subagents: a text classifier, an image analyser, and a policy enforcer. The team notices that the image analyser sometimes directly calls the policy enforcer's action tools to remove posts, bypassing the coordinator. What is the architectural violation and how should it be fixed?

C Merge the image analyser and policy enforcer into a single subagent to simplify the communication flow

Merging subagents with distinct responsibilities (analysis vs enforcement) violates separation of concerns. The image analyser and policy enforcer have fundamentally different roles and tool requirements.

D Add a message queue between the image analyser and the policy enforcer so communication is asynchronous

An async queue between subagents still bypasses the coordinator. In hub-and-spoke, all inter-agent communication must flow through the coordinator regardless of synchronicity.

A Give the image analyser its own copy of the policy enforcer's action tools so it can remove posts itself without calling another subagent.

Duplicating action tools across subagents creates inconsistent enforcement and makes policy changes harder to propagate. The issue is the communication pattern, not tool availability.

B The image analyser breaks hub-and-spoke isolation; scope its tools to image analysis and route all results back through the coordinator.

In hub-and-spoke orchestration, subagents must be isolated and communicate only through the coordinator. The image analyser should return its classification result to the coordinator, which then decides whether to invoke the policy enforcer. Direct subagent-to-subagent communication breaks the architecture.

WHERE THIS COMES FROM

Lesson 1.2: Multi-Agent Orchestration (Isolation principle)

Lesson 1.2: Multi-Agent Orchestration (Hub-and-spoke architecture)

Q21 · D5 5.6 INFORMATION-PROVENANCE / CLAIM-SOURCE-MAPPING q-5-6-003 · Structured Data Extraction Pipeline

CORRECT

A data extraction pipeline extracts financial figures from annual reports. The pipeline cites sources using inline text references like 'According to page 12 of the 2024 Annual Report, revenue was $4.2B.' When the extracted data is consumed by a downstream analytics system, the source attribution is consistently lost. What is the root cause and fix?

B Inline prose citations are fragile. Fix by outputting structured claim-source mappings pairing each value with source, page, and date.

Structured claim-source mappings survive downstream processing because they are data fields, not prose. An analytics system can consume {'value': '4.2B', 'source': '2024 Annual Report', 'page': 12} without losing attribution, whereas it will discard 'According to page 12...' during text processing.

C The downstream system needs to be modified to preserve all input text verbatim without any transformation

Requiring downstream systems to preserve verbatim text is impractical and fragile. Systems legitimately need to transform, aggregate, and restructure data. Attribution should be in a format that survives transformation.

A The downstream system strips text formatting. Fix by making the citations bold or using a special delimiter

The problem is not formatting but data structure. Regardless of text formatting, inline prose citations are lost when text is parsed, summarised, or restructured by downstream systems.

D Add a separate attribution database keyed by value hash that the downstream system queries to look up the source for any extracted figure.

While a database could store attributions, this adds unnecessary infrastructure when the extraction pipeline can simply output structured mappings directly. The attribution should travel with the data, not require a separate lookup.

WHERE THIS COMES FROM

Lesson 5.6: Information Provenance and Multi-Source Synthesis (Structured claim-source mappings)

Q22 · D3 3.4 PLAN-MODE-EXECUTION / MODE-SELECTION q-3-4-001

CORRECT

Your team faces three tasks: (1) restructure a monolith into microservices, (2) fix a null pointer exception in a single function with a clear stack trace, (3) migrate from one logging library to another across 30 files. Which mode should be used for each?

B Plan mode for (1) and (3), direct execution for (2)

Task 1 involves architectural decisions with multiple valid approaches. Task 3 affects 30 files with a pattern that needs design before execution. Task 2 is a single-function fix with a clear stack trace — direct execution is appropriate.

A Plan mode for all three, since they all involve code changes

Task 2 is a well-understood, limited-scope bug fix. Plan mode would waste time for a change where the problem and solution are already clear.

D Direct execution for all three with comprehensive upfront instructions

Task 1 (monolith restructuring) requires codebase exploration and architectural design. Comprehensive upfront instructions assume you already know the right structure without exploring.

C Plan mode for (1), direct execution for (2) and (3)

Task 3 affects 30 files and requires a consistent migration strategy. Without planning, you risk inconsistent application of the logging library change across files.

WHERE THIS COMES FROM

Lesson 3.4: Plan Mode vs Direct Execution (Plan vs direct execution)

Lesson 3.4: Plan Mode vs Direct Execution (Plan mode)

Lesson 3.4: Plan Mode vs Direct Execution (Direct execution)

Q23 · D1 1.3 SUBAGENT-INVOCATION-CONTEXT / PARALLEL-SPAWNING q-1-3-010 ·
Content Moderation and Classification System CORRECT

A user reports a post that contains both potentially defamatory text and an embedded image that may violate graphic content policies. The coordinator receives this report as a single moderation request. What is the correct delegation strategy?

D Create a new combined text-and-image subagent specifically for multi-modal reports

Creating a new combined subagent duplicates capability that already exists in the specialist subagents. The coordinator's role is to decompose and delegate, not to spawn new agents for every combination of concerns.

A Send the entire report to the text classifier first, then forward its output to the image analyser so the image step runs only after the text step has finished.

Sequential processing creates unnecessary latency when the two concerns are independent. The text and image analyses do not depend on each other and can run in parallel.

B Send the entire report to whichever subagent handles the more severe category

Severity cannot be determined until both analyses are complete. Routing to a single subagent also means one concern goes unanalysed.

C Route the text to the text classifier and the image to the image analyser in parallel, then aggregate both results for the final decision.

The coordinator should decompose multi-concern requests into independent subtasks. Text defamation and image policy violations are separate analysis dimensions that can be evaluated in parallel. The coordinator aggregates the results to make a unified moderation decision.

WHERE THIS COMES FROM

Lesson 1.3: Subagent Invocation and Context Passing (Parallel spawning)

Lesson 1.4: Workflow Enforcement and Handoff (Multi-concern request handling)

Q24 · D2 2.1 TOOL-SCHEMA-DESIGN / DESCRIPTIONS q-2-1-005 · Customer Support Resolution Agent CORRECT

A customer support agent has two tools: get_customer_profile (returns name, email, preferences) and get_customer_orders (returns order history with statuses). Users frequently ask 'Tell me about customer X', and the agent calls both tools sequentially, doubling response time. 90% of 'tell me about' queries only need the profile. What is the best approach?

D Set tool_choice to forced selection of get_customer_profile for all turns to prevent unnecessary order lookups.

Forced selection prevents the agent from ever calling get_customer_orders, even when users specifically ask about orders. This solves the over-calling problem by crippling legitimate functionality.

A Consolidate both tools into a single get_customer_info tool that returns all customer data in one call.

Consolidation returns order data on every call, wasting resources for the 90% of queries that only need profile data. It also increases the response payload size unnecessarily.

C Add a system prompt instruction: 'When a user says tell me about, only call get_customer_profile.'

Hard-coding keyword-to-tool mappings in the system prompt is brittle. It does not generalise to other phrasings like 'What do you know about customer X?' and will break if tool names change.

B Refine the descriptions so get_customer_profile is the default and get_customer_orders is order-specific.

Refining tool descriptions with clear defaults is the lowest-effort fix. get_customer_profile should be marked the default for general 'tell me about' queries, directing the agent to call only the profile tool unless order details are explicitly requested.

WHERE THIS COMES FROM

Lesson 2.1: Tool Interface Design (Tool descriptions)

Q25 · D1 1.4 WORKFLOW-ENFORCEMENT-HANDOFF / CONFIDENCE-ROUTING q-1-4-007 · Content Moderation and Classification System CORRECT

The moderation system auto-removes posts classified as policy violations. Production logs reveal that 3% of auto-removed posts were legitimate content (false positives), generating user complaints and eroding platform trust. The team considers two approaches: (A) escalate all borderline cases to human reviewers, or (B) add a confidence threshold where only high-confidence violations are auto-removed and everything else goes to human review. Which approach is better and why?

D Approach B, but apply a single fixed 95% confidence threshold to every category regardless of how costly a false positive is for that content type.

A single fixed threshold across all categories ignores that violation types have different risk profiles. The threshold should be calibrated per category based on the cost of false positives versus false negatives.

C Neither — instead lower the classification threshold so fewer posts are flagged as violations

Lowering the threshold reduces false positives but increases false negatives (genuine violations going undetected). This trades one problem for another rather than adding an appropriate escalation path.

A Approach A is better because human reviewers are always more accurate than automated systems

Escalating all borderline cases overwhelms the human review queue without leveraging the system's ability to handle clear-cut cases autonomously. The goal is to route intelligently, not to defer everything.

B Approach B: auto-remove only high-confidence violations, escalate uncertain cases to human review, and auto-approve high-confidence safe content.

Confidence-based routing creates a tiered system: high-confidence violations are actioned immediately, high-confidence safe content passes through, and uncertain cases receive human review. This balances speed, accuracy, and reviewer workload while directly addressing the false positive problem.

WHERE THIS COMES FROM

Lesson 1.4: Workflow Enforcement and Handoff (Confidence-based escalation)

Lesson 1.4: Workflow Enforcement and Handoff (Enforcement spectrum)

Q26 · D3 3.6 CICD-INTEGRATION / P-FLAG q-3-6-001 · CI/CD Pipeline Integration CORRECT

A CI pipeline script runs 'claude "Analyse this PR"' but the job hangs indefinitely. Logs show Claude Code is waiting for interactive input. What is the correct fix?

A Add the -p flag: claude -p "Analyse this PR"

The -p (--print) flag is the documented way to run Claude Code in non-interactive mode. It processes the prompt, outputs the result to stdout, and exits without waiting for user input.

C Redirect stdin from /dev/null: claude "Analyse this PR" < /dev/null

Unix stdin redirection does not properly address Claude Code's interactive mode. The -p flag is the correct, documented approach.

B Set the environment variable CLAUDE_HEADLESS=true before running the command

CLAUDE_HEADLESS is not a real Claude Code environment variable. This option references a non-existent feature.

D Add the --batch flag: claude --batch "Analyse this PR"

--batch is not a real Claude Code CLI flag. This option references a non-existent feature.

WHERE THIS COMES FROM

Lesson 3.6: CI/CD Integration (-p flag)

Claude Code: Headless / CLI Reference ↗

Q27 · D5 5.5 HUMAN-REVIEW-CALIBRATION / PER-FIELD-CALIBRATION q-5-5-005 · Structured Data Extraction Pipeline

CORRECT

A contract extraction system reports 96% overall accuracy. The team plans to auto-approve all extractions where model confidence exceeds 90%. A pilot reveals party name extraction achieves 99% accuracy but indemnification clause extraction only 71%, even though the model reports high confidence on both. What should they implement before automating?

B Calibrate confidence thresholds per field type using labelled validation sets, and implement stratified sampling to continuously monitor accuracy by document type and field segment.

Field-level confidence calibration using ground truth data exposes the discrepancy between reported confidence and actual accuracy. Stratified sampling provides ongoing monitoring to detect novel error patterns. Together, these ensure automation is only applied where validated accuracy justifies it.

D Train a separate classification model to predict extraction accuracy before deciding whether to automate each extraction.

A separate model adds infrastructure complexity when the solution is to calibrate the existing confidence scores against labelled data. Calibration directly addresses the unreliable confidence outputs without requiring additional ML infrastructure.

C Exclude indemnification clauses from automation entirely and continue automating all other field types at the 90% threshold.

This addresses the known problem but does not detect future calibration issues with other field types. Without systematic calibration and monitoring, other fields with hidden accuracy problems will be automated incorrectly.

A Raise the automation confidence threshold from 90% to 99% to ensure only the most reliable extractions are automated.

Raising the threshold does not fix the calibration problem. The model reports high confidence even for indemnification clauses where it achieves only 71% accuracy. Raw confidence scores are unreliable without calibration.

WHERE THIS COMES FROM

Lesson 5.5: Human Review and Confidence Calibration (Field-level calibration)

Lesson 5.5: Human Review and Confidence Calibration (Stratified sampling)

Q28 · D4 4.4 VALIDATION-RETRY / RETRY-BOUNDARY q-4-4-008 · Structured Data Extraction Pipeline

CORRECT

A validation-retry loop still fails on the same documents after three attempts each. In which situations will further retries not help? (Select 2)

B The required information is simply absent from the source document.

No amount of retrying makes missing information appear; the guide marks absent data as the boundary of retry effectiveness.

C The validation service timed out on the last attempt.

A timeout is a transient failure, which is precisely the class of error a retry is likely to resolve.

A The model placed a correct value in the wrong field on the last attempt.

A value in the wrong field is a semantic validation error; retry-with-error-feedback naming the misplaced field guides the next attempt to correct it.

D The information exists only in an external document that is not provided in context.

Retries cannot reach data outside the model's context; the fix is supplying the external document, not another attempt.

WHERE THIS COMES FROM

Lesson 4.4: Validation, Retry, and Feedback Loops (Retry boundary)

Lesson 4.4: Validation, Retry, and Feedback Loops (Retry with error feedback)

Q29 · D5 5.2 ESCALATION-AMBIGUITY / EXPLICIT-ESCALATION q-5-2-005 · Customer Support Resolution Agent

INCORRECT

A customer says: 'This is ridiculous, I have been waiting 20 minutes. Just connect me to a real person.' The agent has access to the customer's account and can see the issue is a simple password reset that takes 30 seconds to resolve. What should the agent do?

B Escalate to a human agent immediately without attempting to resolve the issue, because the customer explicitly requested a human.

When a customer explicitly says 'connect me to a real person', the agent must escalate immediately. This is one of the three valid escalation triggers. The simplicity of the issue is irrelevant — the customer's explicit request takes priority. (correct answer)

A Acknowledge the frustration and offer to resolve the password reset, explaining it will take only 30 seconds — the issue is straightforward and escalation is unnecessary.

The customer has explicitly requested a human agent. Regardless of how simple the issue is, the agent must honour the explicit request immediately. Attempting to resolve first ignores the customer's stated preference. (your answer)

C Run sentiment analysis to confirm the customer is genuinely frustrated before deciding whether to escalate or attempt resolution.

Sentiment analysis is an unreliable escalation trigger. More importantly, the customer's explicit request for a human is the trigger here, not their emotional state. Sentiment analysis adds unnecessary processing.

D Ask the customer if they would prefer to wait for a human or let the agent resolve the password reset quickly.

The customer has already stated their preference clearly. Asking again ignores their explicit request and adds friction to an already frustrating experience.

WHERE THIS COMES FROM

Lesson 5.2: Escalation and Ambiguity Resolution (Valid escalation triggers)

Lesson 5.2: Escalation and Ambiguity Resolution (Frustration nuance)

Q30 · D2 2.2 STRUCTURED-ERROR-RESPONSES / ACCESS-VS-EMPTY q-2-2-010 · Customer Support Resolution Agent

CORRECT

A customer support agent calls a payment processing tool to check a refund status. The tool returns: {"status": "success", "data": [], "message": "No refunds found for this transaction"}. The agent's recovery logic treats this as an error and retries the request. What is wrong with the agent's behaviour?

B The agent wrongly treats a valid empty result as a failure, not a successful query that found no records.

An empty array result is not a failure when the status indicates success. The tool queried the payment system, found no refund records, and returned that information correctly. The agent should inform the customer that no refund exists for this transaction, not retry.

A The agent should retry more times, as the payment system may be slow to process refund records.

The tool returned status 'success' with a clear message. This is not a slow-processing issue — the query completed successfully and found no refunds. Retrying will produce the same result.

C The tool should not return an empty array — it should return an error with isRetryable: false to prevent the retry.

The tool response is correct — a successful query with no results should return an empty array with a success status. The problem is the agent's recovery logic failing to distinguish valid empty results from actual errors.

D The agent should escalate to a human immediately instead of retrying, as payment queries should never be retried.

There is nothing to escalate. The query succeeded and returned a valid result: no refunds exist. The agent should simply communicate this to the customer.

WHERE THIS COMES FROM

Lesson 2.2: Structured Error Responses (Access failure vs empty result)

Q31 · D1 1.3 SUBAGENT-INVOCATION-CONTEXT / SCOPED-TOOLS q-1-3-013 · Customer Support Resolution Agent CORRECT

An architect is designing a customer support system using the Claude Agent SDK. The system needs a coordinator agent that can delegate to a billing specialist and a technical support specialist. Each specialist needs different tools. How should the architect configure the AgentDefinitions?

A Create one AgentDefinition with all tools from both specialists, and use prompt instructions to tell the agent which tools to use in each context

Combining all tools into one agent violates the principle of scoped tool access. With too many tools, the agent is more likely to use the wrong tool. Each specialist should have only the 4–5 tools relevant to its role.

C Create the specialists as separate API endpoints and have the coordinator call them via HTTP rather than the Task tool

Using HTTP calls instead of the Task tool bypasses the Agent SDK's built-in orchestration capabilities. The Task tool is the designed mechanism for subagent invocation and provides proper context management.

D Give the coordinator all tools and have it handle all requests directly, eliminating the need for subagents

A single agent with all tools has a higher error rate due to tool selection confusion. Specialist subagents with scoped tools produce more reliable results. As a rough guide, aim for a small set of around 4–5 tools per agent.

B Create separate AgentDefinitions for each specialist with scoped tools (4–5 tools each), and give the coordinator the Task tool to spawn them

Each specialist gets its own AgentDefinition with its tools restricted to the 4–5 tools relevant to its role. The coordinator has the Task tool (renamed to Agent in current Claude Code v2.1.63; 'Task' still works as a backward-compatible alias) in its allowedTools so it can spawn specialists. This enforces separation of concerns and scoped tool access.

WHERE THIS COMES FROM

Lesson 1.4: Workflow Enforcement and Handoff (Enforcement spectrum: scoped tools)

Lesson 1.3: Subagent Invocation and Context Passing (Task tool)

Anthropic: Claude Agent SDK Overview ↗

Q32 · D3 3.5 ITERATIVE-REFINEMENT / BATCH-VS-SEQUENTIAL q-3-5-002 INCORRECT

A developer is iterating on a data transformation function with Claude Code and has identified three issues: (1) date parsing mishandles timezone offsets, (2) currency formatting uses the wrong locale, and (3) the output JSON schema has an incorrect field name. Issues 1 and 2 are independent; issue 3 changes the output shape both must conform to. How should the developer provide feedback?

A Send all three issues in a single message to let Claude address them holistically

Batching everything together is appropriate when all issues are interdependent. Here, issues 1 and 2 are independent of each other, and sending them together risks conflating the feedback for those fixes. The interdependency is only between issue 3 and the other two.

B Send each issue in a separate sequential message, waiting for confirmation after each fix

Pure sequential iteration is appropriate when all issues are independent. Here, issue 3 (schema field name) affects the expected output that issues 1 and 2 must conform to. Fixing issues 1 and 2 first without correcting the schema would produce fixes targeting the wrong field name.

D Use the interview pattern to have Claude ask clarifying questions about all three issues before making any changes

The interview pattern is for unfamiliar domains where the developer might miss considerations. Here, the developer has already identified the three specific issues and understands their interdependencies. The question is about feedback sequencing, not domain exploration. (your answer)

C Fix issue 3 (schema field name) first, then address issues 1 and 2 one at a time since they are independent

Issue 3 changes the output schema that issues 1 and 2 must conform to, so it must be resolved first. Once the schema is correct, issues 1 and 2 are independent, so they are fixed one at a time rather than batched, since batching independent issues can confuse which feedback applies to which fix. This follows the principle: resolve dependencies first, batch when fixes interact, and sequence when independent. (correct answer)

WHERE THIS COMES FROM

Lesson 3.5: Iterative Refinement Techniques (Batch vs sequential)

Q33 · D4 4.3 STRUCTURED-OUTPUT / NULLABLE-FIELDS q-4-3-007 · Structured Data Extraction Pipeline CORRECT

An extraction pipeline uses `tool_use` with a 15-required-field schema. For some document types, only 8 of those fields ever appear in the source material. The model consistently fabricates plausible values for the missing 7, and the instruction 'do not fabricate data' has not helped. Should the team (A) keep all fields required and add post-extraction validation, or (B) redesign the schema? Which is correct and why?

C Create separate schemas for each document type, each containing only the fields present in that document type

Multiple schemas add maintenance complexity and require document type classification before extraction. Making fields nullable in a single schema is simpler and equally effective.

A Keep all fields required and add post-extraction validation to detect and remove fabricated values

Post-extraction validation cannot reliably distinguish fabricated values from correct ones — fabricated values are designed to look plausible. The schema itself forces fabrication by making all fields required.

D Switch to prompt-based JSON extraction where you can instruct the model to omit absent fields entirely

Prompt-based JSON sacrifices schema compliance guarantees. tool_use with optional fields gives both flexibility for absent data and schema validation for present data.

B Redesign the schema to make the 7 sometimes-absent fields optional/nullable, allowing the model to return null instead of fabricating

Making fields optional/nullable when source data may be absent removes the schema-level pressure to fabricate. The model can legitimately return null for missing information, which is a correct representation of reality. This addresses the root cause at the schema level.

WHERE THIS COMES FROM

Lesson 4.2: Structured Output with Tool Use (Nullable fields)

Lesson 4.2: Structured Output with Tool Use (Schema does not prevent fabrication)

Q34 · D3 3.1 CLAUDE-MD-HIERARCHY / POSTTOOLUSE-ENFORCEMENT q-3-1-014 · CI/CD Pipeline Integration CORRECT

A fintech company requires that all API response payloads are normalised to snake_case before being logged, and that any file write to the src/api/ directory is automatically linted. They want these enforcements to be deterministic and not rely on the model remembering instructions. Which hook configuration achieves both requirements?

A A PreToolUse hook on file writes that runs the linter before the write completes, and a PostToolUse hook on API response handling that normalises field names to snake_case

PreToolUse fires before the tool runs, so there is no file content to lint yet. Linting must happen after the file is written (PostToolUse). The ordering of hooks to actions is incorrect here.

C Add both requirements to CLAUDE.md with clear examples, and add a PreToolUse hook that reminds the model about these rules before each tool call

CLAUDE.md instructions and PreToolUse reminders both rely on the model's compliance, which is not deterministic. The requirement explicitly states that enforcement must not depend on the model remembering instructions.

B A PostToolUse hook on file writes to src/api/ that runs the linter on the created file, and a PostToolUse hook on data processing that normalises response payloads to snake_case

Both are PostToolUse hooks because both act on completed outputs. The linter runs after a file write to src/api/ is complete, validating the written content. The normalisation hook processes API response data after it is retrieved, converting fields to snake_case before logging. Both provide deterministic enforcement independent of model instructions.

D A PreToolUse hook that blocks any file write to src/api/ unless the file has already been linted, and a PreToolUse hook that blocks API calls unless snake_case normalisation is configured

PreToolUse blocking assumes the file exists before the write, which is contradictory. You cannot lint a file that has not been written yet. PostToolUse hooks that process output after completion are the correct approach for validation and normalisation.

WHERE THIS COMES FROM

Claude Code: Hooks ↗

Q35 · D1 1.1 AGENTIC-LOOPS / MODEL-DRIVEN q-1-1-008 · Content Moderation and Classification System CORRECT

The moderation system's agentic loop uses a hardcoded decision tree: if the classify_content tool returns 'hate_speech', always call escalate_to_human; if it returns 'spam', always call auto_remove. During testing, the team discovers that satirical posts criticising hate speech are being auto-escalated, and sophisticated spam disguised as legitimate marketing slips through. What is the architectural problem?

D Route all ambiguous cases to human review to avoid misclassification

Routing everything ambiguous to humans defeats the purpose of automated moderation. The model can reason about context effectively when given the opportunity; the hardcoded tree is what prevents it.

B Replace the hardcoded decision tree with model-driven decisions, letting Claude weigh the full context of each post before it acts.

Hardcoded decision trees treat classification labels as absolute when real content is nuanced. Letting Claude reason about context (satire vs genuine hate, sophisticated spam patterns) produces better moderation decisions. The agentic loop should let the model decide, not map labels to fixed actions.

C Add a confidence threshold so only high-confidence classifications trigger automatic actions

Confidence thresholds reduce false positives but the fundamental problem remains: a hardcoded tree cannot handle contextual nuance. Low-confidence cases still need model-driven reasoning, not just deferral.

A The classify_content tool needs more granular category labels so it can tell satire criticising hate speech apart from genuine hate speech.

More granular labels do not address the core issue. A hardcoded decision tree cannot adapt to nuance, such as satire versus genuine hate, no matter how fine the labels are. The fix is model-driven reasoning.

WHERE THIS COMES FROM

Lesson 1.1: Agentic Loops (Model-driven decision-making)

Lesson 1.1: Agentic Loops (Anti-patterns)

Q36 · D3 3.1 CLAUDE-MD-HIERARCHY / POSTTOOLUSE-VALIDATION q-3-1-012 · CI/CD Pipeline Integration INCORRECT

A team wants to enforce that all SQL migration files created by Claude Code follow a strict naming convention (YYYYMMDD_HHMMSS_description.sql) and contain a rollback section. Which approach provides deterministic enforcement without relying on the model's judgment?

B Document the naming convention and rollback requirement in the project CLAUDE.md with clear examples

CLAUDE.md instructions rely on the model's interpretation and compliance. While useful as guidance, they do not provide deterministic enforcement. The model may occasionally deviate from the convention, especially under complex prompts.

C Create a .claude/rules/migrations.md file with paths: ["**/migrations/**/*.sql"] containing the naming convention

Rules files provide context-specific instructions but still rely on the model to follow them. They do not provide deterministic enforcement. A hook-based approach guarantees compliance through automated validation. (your answer)

D Use a PreToolUse hook to inject the naming convention into every prompt before file creation occurs

PreToolUse hooks fire before tool execution and are better suited for blocking or modifying tool calls. Injecting instructions into prompts still relies on the model's compliance. PostToolUse validation provides deterministic enforcement by checking the actual output.

A Add a PostToolUse hook on file creation that validates the filename pattern and checks for a rollback section

PostToolUse hooks run deterministic validation code after tool execution. A hook that checks filename patterns and content requirements provides reliable, automated enforcement that does not depend on the model remembering or correctly interpreting naming conventions. (correct answer)

WHERE THIS COMES FROM

Claude Code: Hooks ↗

Q37 · D5 5.2 ESCALATION-AMBIGUITY / AMBIGUOUS-MATCHING q-5-2-004 · Customer Support Resolution Agent CORRECT

A customer support agent searches for a customer named 'Sarah Johnson' and the lookup tool returns three matching accounts with the same name but different addresses and account ages. The agent selects the most recently active account and proceeds with the refund. Later, the customer calls back because the refund was applied to the wrong account. What should the agent have done differently?

A Selected the account with the oldest creation date, since long-standing customers are the most likely callers to a support line.

Account age is an unreliable heuristic for identifying the correct customer. Any selection based on heuristics risks choosing the wrong account.

C Processed the refund across all three accounts to ensure the correct customer received it.

Processing refunds on incorrect accounts creates financial discrepancies and additional work to reverse the erroneous transactions. The agent must identify the correct account first.

D Escalated to a human agent immediately because the system returned ambiguous results.

Ambiguous search results are a routine scenario that the agent can resolve by asking for clarification. Escalation is not warranted when the agent can gather additional information to disambiguate.

B Asked the customer for additional identifying information to disambiguate the matching accounts.

When multiple customers match a search query, the agent must ask for additional identifiers rather than selecting based on heuristics. This ensures the correct account is identified before any action is taken.

WHERE THIS COMES FROM

Lesson 5.2: Escalation and Ambiguity Resolution (Ambiguous customer matching)

Q38 · D1 1.3 SUBAGENT-INVOCATION-CONTEXT / PARALLEL-SPAWNING q-1-3-011 · Content Moderation and Classification System CORRECT

The coordinator receives a batch of 200 flagged posts to moderate. Currently it processes them sequentially, taking 45 minutes. Each post's moderation is independent — the decision on one post does not affect others. How should the coordinator handle this batch?

A Process all 200 posts in a single API call by concatenating them into one large prompt

Concatenating 200 posts into a single prompt causes attention dilution. Quality degrades for posts later in the sequence, and a single failure blocks the entire batch.

D Increase the iteration cap on the agentic loop to allow the agent more time to process all 200 posts

The iteration cap controls how many tool calls the agent can make within a single task, not how many independent tasks it can process. The issue is parallelism, not loop duration.

B Delegate independent posts to parallel subagent instances, with the coordinator aggregating results as they complete

Since each post's moderation is independent, the coordinator should delegate them to parallel instances. This reduces total processing time from sequential (45 minutes) to roughly the time of the slowest individual moderation, and a failure on one post does not block others.

C Split into fixed batches of 20 posts and process each batch sequentially in a single prompt

Batching 20 posts into a single prompt still risks attention dilution across posts within each batch. And sequential batch processing still takes much longer than parallel individual processing.

WHERE THIS COMES FROM

Lesson 1.3: Subagent Invocation and Context Passing (Parallel spawning)

Q39 · D1 1.7 SESSION-STATE-RESUMPTION / FRESH-CONTEXT q-1-7-008 · Content Moderation and Classification System CORRECT

The moderation system handles appeals where users contest a moderation decision. Currently, the same agent that made the original decision re-evaluates the appeal. Appeal overturn rates are suspiciously low. What is the most effective architectural change?

D Automatically overturn decisions where the user provides any appeal justification to improve user trust

Automatic overturn on any appeal completely undermines moderation. Users who genuinely violated policies would simply appeal every decision.

A Add stronger instructions to the appeal handler's prompt requiring it to weigh the user's perspective fairly and set aside its earlier decision.

Prompt instructions cannot overcome the bias of an agent reviewing its own decision. The original reasoning context still influences the re-evaluation regardless of instructions.

C Route appeals to a separate agent instance that cannot see the original reasoning, given only the content and the user's justification.

A fresh agent instance without access to the original reasoning context evaluates the content independently. This avoids confirmation bias from the original decision. The appeal agent sees only the content and the user's argument, enabling a genuinely independent review.

B Route appeals to a human reviewer who has full authority to overturn automated decisions

Human review for every appeal does not scale and removes the benefit of automated moderation. Some appeals can be resolved automatically by a separate, unbiased instance.

WHERE THIS COMES FROM

Lesson 1.7: Session State and Resumption (Stale context)

Lesson 1.2: Multi-Agent Orchestration (Isolation principle)

Q40 · D2 2.4 MCP-SERVER-INTEGRATION / DESCRIPTIONS q-2-4-002 · Customer Support Resolution Agent INCORRECT

A customer support team configures an MCP server for their internal CRM system. The agent frequently ignores the CRM MCP tool and instead uses the built-in Grep tool to search local log files for customer information, producing incomplete results. The MCP tool's description reads: 'CRM tool'. What should the team do first?

D Expand the sparse 'CRM tool' description to detail its full customer records and structured output.

When an MCP tool's description is sparse, the agent defaults to familiar built-in tools like Grep. Enhancing the description to explain its full customer records, transaction history, account status, and structured output makes the agent prefer it for relevant queries. This is the recommended fix for MCP tool underutilisation. (correct answer)

A Remove the Grep tool from the agent's available tools so it cannot fall back to local file search.

Removing Grep cripples the agent for legitimate file search tasks. The problem is not that Grep exists but that the MCP tool's description does not communicate its superior capabilities for customer data.

B Add a system prompt instruction: 'Always use the CRM tool for customer queries. Never use Grep for customer data.'

Hard-coding tool preferences in the system prompt is brittle and does not scale. If new tools are added or the CRM tool is renamed, the instruction becomes stale or causes conflicts. (your answer)

C Move the MCP server configuration from .mcp.json to ~/.claude.json so it takes higher priority.

Configuration scoping (project vs user level) does not affect tool selection priority. Both levels make tools available simultaneously. The issue is the tool description, not the configuration location.

WHERE THIS COMES FROM

Lesson 2.4: MCP Server Integration (Enhancing MCP descriptions)

Q41 · D2 2.1 TOOL-SCHEMA-DESIGN / DESCRIPTIONS q-2-1-001 · Customer Support Resolution Agent CORRECT

Production logs show an agent frequently calls get_customer when users ask about orders (e.g. 'check my order #12345'), instead of calling lookup_order. Both tools have minimal descriptions ('Retrieves customer information' / 'Retrieves order details') and accept similar identifier formats. What is the most effective first step to improve tool selection reliability?

D Consolidate both tools into a single lookup_entity tool that accepts any identifier and internally determines which backend to query.

Consolidation is a valid architectural choice but requires significantly more effort than expanding descriptions. The exam favours proportionate first steps.

B Expand each tool's description to include input formats, example queries, edge cases, and boundaries explaining when to use it versus similar tools.

Tool descriptions are the primary mechanism LLMs use for tool selection. Expanding them is the lowest-effort, highest-leverage fix that directly addresses the root cause.

C Implement a routing layer that parses user input before each turn and pre-selects the appropriate tool based on detected keywords.

A routing layer is over-engineered as a first step. It bypasses the LLM's natural language understanding and adds infrastructure complexity.

A Add 5-8 few-shot examples to the system prompt demonstrating correct tool selection patterns for order-related queries.

Few-shot examples add token overhead without fixing the underlying issue. The root cause is that descriptions do not differentiate the tools.

WHERE THIS COMES FROM

Lesson 2.1: Tool Interface Design (Tool descriptions)

Lesson 2.1: Tool Interface Design (Misrouting)

Anthropic: Tool Use Documentation ↗

Q42 · D4 4.4 VALIDATION-RETRY / RETRY-WITH-ERROR q-4-4-007 · Structured Data Extraction Pipeline CORRECT

Your extraction pipeline has a validation step that checks extracted financial data. When validation fails, the system retries by sending just the original document with the same prompt. Retry success rates are below 10%. The most common failure is the model placing the 'net amount' value in the 'gross amount' field and vice versa. How should the retry be restructured?

D Switch to a different model for the retry, since the original model has a persistent field-mapping bias

The issue is the absence of error context in the retry, not a model-specific bias. Any model retrying without seeing the specific validation error will have a low success rate.

C Add a post-processing rule that automatically swaps net and gross amounts when validation detects the pattern

Hard-coded swap rules are brittle and mask the underlying extraction error. They also fail if the issue manifests differently in other documents. Error feedback teaches the model to extract correctly.

A Increase retries from 1 to 5, since more attempts at the same prompt will eventually produce the correct field mapping by chance

If the model consistently maps fields incorrectly with the same prompt, more retries without feedback will produce the same incorrect mapping. The model needs to see its specific error.

B Send the original document, the failed extraction, and the specific validation error naming the swapped fields.

Retry-with-error-feedback is dramatically more effective than naive retries. The model seeing its specific mistake (net and gross amounts swapped) and the validation error allows targeted self-correction rather than repeating the same misinterpretation.

WHERE THIS COMES FROM

Lesson 4.4: Validation, Retry, and Feedback Loops (Retry with error feedback)

Lesson 4.4: Validation, Retry, and Feedback Loops (Self-correction flow)

Q43 · D1 1.5 AGENT-SDK-HOOKS / PRETOOLUSE-ENFORCEMENT q-1-5-001 · Customer Support Resolution Agent CORRECT

An agent occasionally processes international transfers without required compliance checks. The compliance team requires 100% enforcement of anti-money laundering (AML) checks before any international transfer. What is the correct approach?

B Implement a PreToolUse hook that blocks transfer execution until AML verification returns a pass result

A hook intercepts the outgoing tool call before execution and physically blocks it until the compliance check passes. This provides the deterministic guarantee that AML rules require.

C Add a PostToolUse hook to flag completed transfers that skipped AML checks

PostToolUse hooks run after execution. By the time the hook fires, the non-compliant transfer has already been processed. Prevention is required, not detection.

D Train the agent with few-shot examples showing the correct AML verification workflow

Few-shot examples improve accuracy but do not guarantee 100% compliance. Regulatory requirements demand deterministic enforcement via hooks.

A Add detailed AML check instructions to the system prompt with examples of correct behaviour

Prompt instructions provide probabilistic compliance. With international transfers and AML regulations, a single failure could result in legal penalties. 100% enforcement requires deterministic guarantees.

WHERE THIS COMES FROM

Lesson 1.5: Agent SDK Hooks (PreToolUse hooks)

Anthropic: Agent SDK Hooks ↗

Q44 · D5 5.5 HUMAN-REVIEW-CALIBRATION / AGGREGATE-METRICS-TRAP q-5-5-001 · Structured Data Extraction Pipeline CORRECT

A structured data extraction system achieves 97% overall accuracy across all document types. The team proposes automating all extractions where model confidence exceeds 95% to reduce human review costs. What is the critical risk in this approach?

C The model will become overconfident over time as it processes more documents, requiring regular retraining

LLMs do not train during inference. The issue is existing calibration gaps, not drift.

A The 95% confidence threshold is too low and should be raised to 99% for automation

The threshold value is not the core issue — the problem is that aggregate metrics hide per-type performance disparities.

B Aggregate accuracy can mask poor per-type performance, and confidence scores need calibration before use.

97% overall can hide 40% error rates on specific document types. Without stratified validation and confidence calibration, automation will silently fail on certain inputs.

D Automated extractions should always have human review regardless of confidence, making the proposal fundamentally flawed

Automation is valid when properly validated — the issue is doing it based on uncalibrated aggregate metrics, not the concept of automation itself.

WHERE THIS COMES FROM

Lesson 5.5: Human Review and Confidence Calibration (Aggregate metrics trap)

Lesson 5.5: Human Review and Confidence Calibration (Confidence calibration)

Q45 · D3 3.3 PATH-SPECIFIC-RULES / MULTI-RULESET q-3-3-002 · CI/CD Pipeline Integration INCORRECT

A DevOps team's polyglot codebase has Terraform files in terraform/, Kubernetes manifests in k8s/, and Dockerfiles scattered throughout various service directories. They want Claude Code to automatically apply infrastructure-specific conventions when editing each type of file, without loading all conventions for every session. What is the correct approach?

C Create a single .claude/rules/infrastructure.md file with paths: ["**/*"] containing all infrastructure conventions

The glob pattern **/* matches every file in the codebase, which means the infrastructure conventions would load for every session — exactly the same as putting them in root CLAUDE.md. This defeats the purpose of conditional loading. (your answer)

B Add all infrastructure conventions to the root CLAUDE.md with clear section headings so the model knows which section applies

Root CLAUDE.md loads for every session. All infrastructure conventions would consume tokens even when editing TypeScript application code. Section headings do not prevent the conventions from being loaded — the model still processes the entire file.

A Create three directory-level CLAUDE.md files: one in terraform/, one in k8s/, and one in the project root for Docker conventions

Directory-level CLAUDE.md files in terraform/ and k8s/ would work for those specific directories, but Dockerfiles are scattered throughout various service directories. A single root CLAUDE.md for Docker conventions would load for every session, defeating the token efficiency goal. This approach does not handle the cross-directory Dockerfile pattern.

D Create three path-scoped .claude/rules/ files (terraform.md, kubernetes.md, docker.md), each targeting its own file types

Each rule file targets specific infrastructure file types with appropriate glob patterns. Terraform conventions load only when editing Terraform files, Kubernetes conventions only for k8s manifests, and Docker conventions only for Dockerfiles — regardless of which directory the Dockerfiles are in. This provides token-efficient conditional loading for all three file types. (correct answer)

WHERE THIS COMES FROM

Lesson 3.3: Path-Specific Rules (Multiple rule files)

Lesson 3.3: Path-Specific Rules (Glob patterns)

Q46 · D2 2.2 STRUCTURED-ERROR-RESPONSES / BUSINESS-ERROR q-2-2-002 · Customer Support Resolution Agent CORRECT

A support agent tries to refund $850, but policy caps automated refunds at $500. The MCP tool returns 'Operation failed' as its error. The agent retries three times before escalating to a human. What change to the tool's error response would most effectively prevent this behaviour?

D Catch the error in the agent's system prompt with an instruction: 'Never retry refund operations that fail'.

Hard-coding retry rules per operation in the system prompt is brittle and does not scale. The proper fix is structured error metadata that the agent's recovery logic can interpret programmatically for any tool.

C Return a structured error with errorCategory: 'business', isRetryable: false, and a customer-friendly description explaining the policy limit.

Business errors are never retryable — retrying a policy violation will always fail. Structured metadata with errorCategory and isRetryable: false tells the agent to stop retrying and take an alternative path, such as offering partial refund or escalating to a supervisor.

A Include a longer error message explaining the policy: 'Operation failed because the refund amount of $850 exceeds the $500 automated refund limit'.

A better message helps human readers but does not give the agent structured metadata to determine that retrying is futile. The agent needs machine-readable fields, not just better prose.

B Set the MCP isError flag and add a retry-after header so the agent waits before retrying.

A retry-after header implies the error is transient and will resolve with time. A business policy violation is never transient — no amount of waiting will change the refund limit. This would cause delayed retries that still fail.

WHERE THIS COMES FROM

Lesson 2.2: Structured Error Responses (Error categories)

Q47 · D5 5.1 CONTEXT-WINDOW-MANAGEMENT / FACTS-BLOCK q-5-1-001 · Customer Support Resolution Agent CORRECT

A customer support agent handles a multi-issue session. After several turns, the agent refers to 'your recent refund request' instead of the specific $247.83 refund for order #8891. The conversation history is being summarised between turns to manage context length. What is the most effective fix?

A Increase the context window size to avoid summarisation entirely

This postpones the problem but does not solve it — eventually the context will fill, and summarisation will still destroy specifics.

B Extract transactional facts into a persistent case facts block outside the summarised history.

This directly addresses the progressive summarisation trap by ensuring critical numerical and transactional data is never condensed.

D Store the full conversation in an external database and retrieve relevant turns on demand

This adds infrastructure complexity without addressing the core issue of which facts must persist in every prompt.

C Instruct the model to preserve every amount, date, and order number verbatim whenever it summarises earlier turns.

Prompt-based instructions for summarisation are unreliable — the model will still compress details probabilistically.

WHERE THIS COMES FROM

Lesson 5.1: Context Window Management (Progressive summarisation)

Lesson 5.1: Context Window Management (Persistent facts)

Q48 · D4 4.4 VALIDATION-RETRY / PYDANTIC-VALIDATION q-4-4-010 · Structured Data Extraction Pipeline CORRECT

The pipeline validates each tool_use extraction against a Pydantic model with model_validate(). Validation succeeds and every field has the correct type, yet extracted totals still disagree with the documents' stated totals. What explains this, and what is the fix?

D Semantic disagreement indicates hallucination; lower the temperature so extraction becomes deterministic

Sampling settings do not create correctness, and determinism would only repeat the same wrong extraction; this failure class belongs to semantic validation.

B Type checks passed but no rule relates the totals; add a model validator for the sum rule and route its errors into retries

Schema conformance and semantic correctness are different layers; the sum relationship must be written as a validator, whose errors drive the retry loop.

A model_validate() is failing silently; pin an older Pydantic version until the regression is fixed

model_validate() did its job: the extraction conforms to the model's types. Nothing failed silently; the missing check was never defined.

C The schema needs numeric minimum and maximum constraints on the stated_total field so that mismatched totals are rejected at parse time

A per-field range cannot express a relationship between two fields; the mismatch is only detectable by comparing them in validation code.

WHERE THIS COMES FROM

Lesson 4.4: Validation, Retry, and Feedback Loops (Pydantic as the validation layer)

Q49 · D5 5.1 CONTEXT-WINDOW-MANAGEMENT / TOKEN-BUDGET q-5-1-003 · Customer Support Resolution Agent CORRECT

A customer support agent uses a token budget of 200k tokens. The system prompt consumes 8k tokens, conversation history takes 120k tokens, and the most recent tool call result returned 65k tokens. The agent is struggling to produce thorough responses. What is the most likely cause and fix?

C The model needs a higher max_tokens parameter to produce longer responses

max_tokens caps the response length but cannot exceed the remaining context budget. If 193k of 200k is consumed by input, increasing max_tokens beyond 7k would have no effect.

B The system prompt is too large at 8k tokens and should be reduced to under 2k to leave more room for the response

The system prompt at 8k is a small fraction of the budget. Reducing it by 6k would help marginally but misidentifies the problem — conversation history and tool results are consuming the vast majority of the budget.

D The conversation history should be cleared entirely to give the model a fresh context for each response

Clearing history would free tokens but destroy conversation continuity. The customer would need to repeat their issue. Selective summarisation is far preferable to wholesale deletion.

A Input already consumes 193k of the 200k budget. Summarise history or trim verbose tool results.

Token budget is shared: system prompt + conversation history + tool results + available for response = total budget. With 193k consumed, only 7k remains for the response. The fix targets the largest movable allocations: summarise older history or trim verbose tool output.

WHERE THIS COMES FROM

Lesson 5.1: Context Window Management (Tool result trimming)

Lesson 5.1: Context Window Management (Full conversation history)

Q50 · D2 2.3 TOOL-DISTRIBUTION-CHOICE / ROLE-SCOPING q-2-3-002 CORRECT

A developer productivity platform has a code review agent equipped with 18 tools covering code analysis, documentation lookup, test generation, dependency checking, security scanning, and deployment validation. Reviews are slow and the agent frequently selects the wrong tool. What is the most effective architectural change?

B Implement a routing classifier that pre-selects the appropriate tool before each agent turn.

A routing classifier adds infrastructure complexity and bypasses the LLM's natural language understanding. The root cause is too many tools, not inadequate routing.

C Switch tool_choice from 'auto' to 'any' so the model is forced to select a tool on every turn.

Forcing tool selection does not help when the agent cannot reliably choose among 18 options. The problem is selection accuracy due to overload, not whether the model calls a tool at all.

A Rewrite all 18 tool descriptions with explicit boundaries and 'use this when' guidance so the agent can tell them apart.

Better descriptions help but do not solve the fundamental problem: 18 tools exceed the recommended range and create too much decision complexity. Selection accuracy degrades regardless of description quality when tool count is too high.

D Split the agent into role-specific agents with 4-5 tools each, scoped to their specialisation.

The recommended range is 4-5 tools per agent. Splitting into role-specific agents (for example separate agents for code analysis, security scanning, and test generation) eliminates decision complexity and prevents cross-specialisation misuse. Each agent excels within its narrow remit.

WHERE THIS COMES FROM

Lesson 2.3: Tool Distribution and Tool Choice (Role-specific scoping)

Lesson 2.3: Tool Distribution and Tool Choice (Tool overload)

Q51 · D4 4.3 STRUCTURED-OUTPUT / TOOL-CHOICE-FORCING q-4-3-006 · Structured Data Extraction Pipeline CORRECT

Your data extraction pipeline uses tool_use with tool_choice set to 'auto'. You need guaranteed structured output for every request, but some documents occasionally produce conversational text responses instead. You have only one extraction tool defined. What is the most reliable fix?

B Set tool_choice to force the specific extraction tool by name

Setting tool_choice to {type: 'tool', name: 'extract_data'} forces the model to call that specific tool on every request, guaranteeing schema-compliant structured output with no possibility of text-only responses.

A Add a system prompt instruction: 'You must always use the extraction tool and never produce text responses'

System prompt instructions cannot guarantee tool use when tool_choice is 'auto'. The API-level tool_choice parameter is the authoritative mechanism.

C Add a post-processing step that retries any request returning text instead of a tool call

Retrying with the same 'auto' tool_choice may produce the same text response. Fixing tool_choice eliminates the problem at the API level rather than patching it after the fact.

D Switch to prompt-based JSON extraction with strict formatting instructions for more predictable output

Prompt-based JSON is less reliable than tool_use. It risks syntax errors and missing fields. Forcing the tool via tool_choice is the correct approach.

WHERE THIS COMES FROM

Lesson 4.2: Structured Output with Tool Use (tool_choice modes)

Anthropic: Tool Use Documentation ↗

Q52 · D1 1.1 AGENTIC-LOOPS / TOOL-RESULT-HANDLING q-1-1-009 · Customer Support Resolution Agent CORRECT

A customer-support agent runs an agentic loop. Each turn it inspects `stop_reason`; when the value is `"tool_use"` it executes the requested tool and receives the tool's output. Before it calls the model again so the loop can continue coherently, what must the agent do with that output?

B Replace the previous assistant message with the tool output to keep the context window small

Overwriting the assistant message discards the tool_use block that the result corresponds to, breaking the tool_use/tool_result pairing the API requires and losing the reasoning that led to the call.

A Append the tool result to the conversation history as a new message, then send the full updated conversation on the next call.

The Messages API is stateless, so every call must carry the whole conversation. The tool result is appended as a new message paired with the tool_use block that requested it, which is how the model sees what its call returned and decides the next step.

C Send only the tool output on the next call, since the API retains the earlier turn server-side and will pair it with the pending tool request.

The Messages API keeps no server-side memory of the turn. Sending only the tool output drops all prior context, so the model cannot continue the task coherently.

D Store the tool output in an external store and pass a reference id to the model on the next call

The model cannot dereference an external id. The tool result has to be present in the conversation the model actually receives.

WHERE THIS COMES FROM

Lesson 1.1: Agentic Loops (Tool result handling)

Anthropic: Claude Agent SDK Overview ↗

Q53 · D3 3.4 PLAN-MODE-EXECUTION / HYBRID q-3-4-005 CORRECT

A team is debugging a complex distributed system issue. The lead developer wants to use Claude Code to explore logs, trace request flows, and form hypotheses without making any changes to the codebase. Partway through the investigation, they identify a one-line fix in a configuration file and want to apply it immediately. What is the optimal workflow?

D Use allowedTools to restrict to read-only tools for investigation, then start a new session with write permissions for the fix

Starting a new session for the fix would lose all the investigation context (log analysis, request tracing, hypothesis formation) that led to identifying the fix. Switching modes within the same session preserves this valuable context.

B Use direct execution throughout, with detailed upfront instructions describing both the investigation process and potential fix patterns

Direct execution is inappropriate for the investigation phase because the problem and solution are not yet understood. Providing detailed upfront instructions assumes knowledge of the issue that the investigation is meant to discover.

A Start in plan mode for investigation, then switch to direct execution for the one-line fix once identified

Plan mode is ideal for the investigation phase: exploring logs, tracing flows, and evaluating multiple hypotheses without committing to changes. Once the fix is identified and well-understood (a clear-scope, single-file change), switching to direct execution applies it efficiently without unnecessary planning overhead.

C Use plan mode for the entire session, including the one-line fix, to maintain consistency

While plan mode is correct for investigation, continuing to use it for a well-understood one-line fix adds unnecessary overhead. Once the fix is identified and scoped, direct execution is the appropriate mode for a clear, limited change.

WHERE THIS COMES FROM

Lesson 3.4: Plan Mode vs Direct Execution (Hybrid plan then execute)

Q54 · D5 5.5 HUMAN-REVIEW-CALIBRATION / STRATIFIED-SAMPLING q-5-5-009 · Structured Data Extraction Pipeline CORRECT

Your extraction pipeline reports 97% aggregate accuracy, and you want to automate high-confidence extractions. Which safeguards should be in place before you reduce human review? (Select 3)

B Rely on the aggregate accuracy figure once it has held steady for a full quarter.

Stability of the aggregate number does not reveal per-segment weaknesses, which is the risk the guide highlights.

E Expand the review team until every extraction can be checked by hand.

Reviewing everything ignores the point of calibration: routing limited reviewer capacity to low-confidence and ambiguous cases.

C Verify accuracy separately by document type and field segment, not just in aggregate.

A strong aggregate figure can mask poor performance on specific document types or fields.

A Calibrate field-level confidence thresholds against a labelled validation set.

Confidence scores only route review attention correctly once they are calibrated on labelled data.

D Keep stratified random samples of high-confidence extractions under review to measure error rates and catch novel patterns.

Stratified sampling is the guide's mechanism for ongoing measurement after automation.

WHERE THIS COMES FROM

Lesson 5.5: Human Review and Confidence Calibration (Aggregate metrics trap)

Lesson 5.5: Human Review and Confidence Calibration (Stratified sampling)

Lesson 5.5: Human Review and Confidence Calibration (Field-level calibration)

Q55 · D1 1.1 AGENTIC-LOOPS / LIFECYCLE q-1-1-001 · Customer Support Resolution Agent CORRECT

A developer's customer support agent sometimes terminates prematurely because they check if response.content[0].type == 'text' to determine completion. The agent stops even when Claude has more tools to call. What is the bug and how should they fix it?

B Check the stop_reason field instead of content type: continue on 'tool_use', terminate on 'end_turn'.

The stop_reason field is the deterministic, authoritative signal for loop control. Claude can return text alongside tool_use blocks, so checking content type is unreliable.

A Add an iteration cap of 15 loops and treat reaching the cap as the signal that the agent has finished its work.

Arbitrary caps do not address the root cause, and using the cap as a completion signal is itself an anti-pattern. The agent exits early because it misreads the response type; the fix is to inspect stop_reason, not to cap iterations.

C Parse the assistant's text for phrases like 'I have completed' before terminating

Natural language parsing is ambiguous and unreliable. The stop_reason field already provides an unambiguous signal.

D Set tool_choice to 'any' so Claude always calls a tool instead of returning text

This forces tool use even when the agent is genuinely finished, creating an infinite loop rather than fixing the detection logic.

WHERE THIS COMES FROM

Lesson 1.1: Agentic Loops (Agentic loop lifecycle)

Anthropic: Messages API Reference ↗

Q56 · D2 2.1 TOOL-SCHEMA-DESIGN / BOUNDARY-DESCRIPTIONS q-2-1-006 · Customer Support Resolution Agent CORRECT

A platform has `search_knowledge_base` ('Searches help articles') and `process_action` ('Handles customer actions like refunds and plan changes'). The agent picks `search_knowledge_base` for 'cancel my subscription' requests. After adding 'subscription cancellations' to `process_action`'s description, it now picks `process_action` for knowledge queries like 'how does cancellation work?'. What is the most effective solution?

C Consolidate search_knowledge_base and process_action into a single tool that determines the correct action based on user intent.

Consolidation pushes the routing problem inside the tool implementation and loses the LLM's natural language understanding. Keeping tools separate with clear boundary descriptions is simpler and more maintainable.

D Add few-shot examples to the system prompt showing five cancellation scenarios with the correct tool for each.

Few-shot examples consume tokens and do not scale to the many variations of cancellation queries. Boundary descriptions in tool definitions are more efficient and generalise better.

B Remove the word 'cancellation' from both tool descriptions to eliminate the keyword conflict entirely.

Removing relevant keywords makes both tools less discoverable for cancellation-related queries. The solution is to differentiate when each tool handles cancellation, not to hide the concept from both.

A Add boundary descriptions to both tools that separate executing a cancellation from learning how cancellation works.

Boundary descriptions resolve the ambiguity by intent. process_action should say to use it when the customer wants to execute a cancellation; search_knowledge_base should say to use it when the customer wants to learn about the process without taking action. Stating both boundaries prevents cross-routing in either direction.

WHERE THIS COMES FROM

Lesson 2.1: Tool Interface Design (Boundary descriptions)

Lesson 2.1: Tool Interface Design (Misrouting)

Q57 · D1 1.4 WORKFLOW-ENFORCEMENT-HANDOFF / MULTI-CONCERN q-1-4-003 · Customer Support Resolution Agent

INCORRECT

A financial services company's customer support agent receives a request: 'I was charged twice for my subscription, and I also need to update my billing address.' The agent processes the address change successfully but only partially investigates the double-charge, providing an incomplete resolution. What architectural pattern should be applied to handle this correctly?

B Route the request to two separate specialised agents — one for billing disputes and one for address changes

Splitting the request across agents adds coordination overhead and may lose the shared context between the two issues. The correct pattern is multi-concern decomposition within the agent's workflow, investigating each issue in parallel with shared context. (your answer)

A Add a system prompt instruction telling the agent to address all customer concerns before responding

Prompt-based guidance has a non-zero failure rate. Multi-concern decomposition is an architectural pattern, not a prompting issue. The agent needs a structured approach to identifying and tracking multiple concerns.

C Decompose the request into items, investigate each in parallel with shared context, then synthesise a unified resolution.

Multi-concern request decomposition breaks the compound request into distinct items, investigates each in parallel using shared context (the customer's account), and synthesises a unified resolution that addresses all concerns. This ensures no concern is dropped or partially addressed. (correct answer)

D Process the concerns sequentially — complete the address change first, then investigate the double-charge in a follow-up interaction

Sequential processing across separate interactions is a poor customer experience and risks the second concern being lost. The agent should decompose, investigate in parallel, and provide a unified resolution in a single interaction.

WHERE THIS COMES FROM

Lesson 1.4: Workflow Enforcement and Handoff (Multi-concern request handling)

Q58 · D4 4.4 VALIDATION-RETRY / INTER-STEP-VALIDATION q-4-4-006 · Structured Data Extraction Pipeline INCORRECT

Your document processing pipeline chains three steps: (1) extract raw text, (2) classify document type, (3) extract structured fields based on type. Step 2 occasionally misclassifies invoices as purchase orders, causing step 3 to extract the wrong fields. The team proposes combining steps 2 and 3 into a single prompt to reduce errors. What is the better approach?

D Run step 2 three times and use majority voting to determine the document type

Majority voting adds cost and latency without addressing the root cause. Validation between steps is more efficient and catches the specific failure mode.

C Add more few-shot classification examples to step 2's prompt so misclassifications stop occurring

Better examples may help, but without validation between steps, misclassifications still cascade silently into step 3. Inter-step validation is the structural fix. (your answer)

A Combine steps 2 and 3 as proposed, since fewer steps means fewer failure points

Combining steps creates a less focused prompt that must handle both classification and extraction simultaneously. This increases attention dilution and makes it harder to diagnose which part fails.

B Keep steps separate and validate the classification between steps 2 and 3 before extraction

Keeping steps separate maintains focused prompts. Adding validation between steps catches misclassifications before they cascade into wrong-field extraction. This is the core principle of prompt chaining: focused steps with inter-step validation. (correct answer)

WHERE THIS COMES FROM

Lesson 4.4: Validation, Retry, and Feedback Loops (Self-correction flow)

Lesson 4.4: Validation, Retry, and Feedback Loops (Retry boundary)

Q59 · D4 4.6 MULTI-PASS-REVIEW / MULTI-PASS q-4-6-004 · CI/CD Pipeline Integration CORRECT

Your CI/CD code review system analyses pull requests averaging 8-12 files. Reviews are thorough on the first few files but become increasingly superficial on later files, sometimes missing obvious issues. What architectural change best addresses this pattern?

A Randomise the file order before each review run so a different subset of files receives the thorough early-pass attention each time.

Randomising order means different files get superficial treatment each time, but does not ensure all files receive thorough review. The root cause — attention dilution — remains.

B Increase the model's context window to give it more space to analyse all files

Attention dilution is not a context window size problem. The model has enough space but distributes attention unevenly across many files.

D Add a second full-PR review pass and merge findings from both passes

A second pass on the full PR suffers from the same attention dilution. Both passes will likely be thorough on early files and superficial on later ones.

C Split the review into per-file passes so each file gets dedicated attention, then a cross-file integration pass.

Per-file analysis ensures each file receives consistent, thorough attention. The cross-file integration pass then catches issues that span multiple files, such as inconsistent interfaces or data flow problems. This directly solves attention dilution.

WHERE THIS COMES FROM

Lesson 4.6: Multi-Instance Review and Output Validation (Multi-pass review)

Lesson 4.6: Multi-Instance Review and Output Validation (Why context windows do not fix it)

Q60 · D3 3.6 CICD-INTEGRATION / OUTPUT-FORMAT q-3-6-004 · CI/CD Pipeline Integration CORRECT

A team wants their CI pipeline to run Claude Code for both PR code review and automated test generation. The PR review should output structured JSON for integration with their review dashboard, while the test generation should produce standard text output. How should the two pipeline steps be configured?

A Run both steps with -p and configure the review dashboard to parse plain text output from both steps

Plain text output requires fragile parsing logic in the review dashboard. The --output-format json flag provides structured, machine-readable output that is far more reliable for programmatic integration.

B Run the review step with -p --output-format json and the test generation step with -p only, as separate non-interactive invocations

Each step uses -p for non-interactive mode. The review step adds --output-format json for structured output that the dashboard can parse reliably. The test generation step uses default text output since it produces code files, not structured data. Running them as separate invocations ensures session context isolation.

C Run both steps in a single Claude Code session with -p, using different prompts to request different output formats

Running both steps in a single session violates session context isolation. The review step would retain reasoning context from test generation, reducing review effectiveness. Each step should be an independent invocation.

D Run both steps with --output-format json and have the test generation step extract code from the JSON response

While this would work technically, it adds unnecessary complexity to the test generation step. Using JSON output format only for the step that needs structured parsing (review) is the cleaner approach.

WHERE THIS COMES FROM

Lesson 3.6: CI/CD Integration (Structured output)

Lesson 3.6: CI/CD Integration (-p flag)

Claude Code: Headless / CLI Reference ↗